ScrumDesk Implementation Playbook

Smart Refrigerator Manager | Release 2

A practical, step-by-step guide for translating this enhancement into a ScrumDesk project: project goal, epics, stories, acceptance criteria, sprint planning, task assignment, daily tracking, release evidence, and assessment close-out.

Project	Details
Product	Existing Smart Refrigerator Manager web app (Spring Boot / Thymeleaf)
Release goal	English/Japanese UX, waste history, inventory-based AI menu ideas, and account/profile essentials
Prepared	10 November 2026
Named members	Sabind Daryal (all coding); Harsha (surname / student ID to confirm)
Submission deadline	22 December 2026, 23:59, per supplied CN7021 brief
Board state	Guide and backlog specification prepared; live ScrumDesk workspace not connected or verified

Use this as the source of truth for entering cards. Keep unfinished work open until acceptance evidence exists. A code change by itself does not make a story Done.

Important course constraint

The supplied brief specifies groups of three or four. Only Sabind and Harsha were named. Confirm the official group roster and any additional member(s) with the tutor; do not fabricate names, hours, signatures, or ScrumDesk activity. The proposed allocation keeps Sabind as the sole coder and assigns Harsha non-coding requirements, board, acceptance-evidence, minutes, and report work.

1. Release goal, scope and evidence rule

Product goal: Help a signed-in household user understand what is in the fridge, act on food nearing expiry, record what is discarded, and get meal ideas from current stock in English or Japanese.

Release goal: Ship the web enhancement with complete locale handoff from the login screen into the dashboard; consistently localized labels, placeholders, validation and dynamic text; a polished Apple-inspired responsive UI; user-owned waste history; inventory-aware menu suggestions with an optional configured AI provider and safe local fallback; and account/profile/recovery flows. Flutter QR scanning stays outside this release.

In scope

- Locale choice on login/signup/recovery is carried through successful login; after login, it is saved as the account language. Profile changes persist and survive a later sign-in.
- English and Japanese cover visible UI, HTML placeholders, accessibility labels, form errors, category/status labels, dates, dynamic empty states, waste history labels, and menu text. User-entered food names and reasons remain user content.
- Waste actions create owner-scoped history and update active stock transactionally. Users can view history and total discarded quantity.
- Menu service matches local inventory, considers expiry where available, requests output in the user language when an external provider is explicitly configured, and falls back locally when no key or provider failure.
- Account scope includes email-based registration, profile name/language, and single-use time-limited email reset codes. Flutter/mobile work is excluded.

Board status discipline

At guide creation, code for several baseline features already exists, but current language handoff and visual redesign are being completed, and end-to-end acceptance tests are still required. Create or update cards as In Progress or Review, then move to Done only after the acceptance criteria below pass and the evidence is attached. No live board, branch history, CI run, or ScrumDesk screenshot has been verified.

2. Create and configure the ScrumDesk project

Step	Action in ScrumDesk	Value / output
1	Create a new project	Name: Smart Refrigerator Manager - Release 2. Description: continuation of existing open-source web app; release scope is this guide.
2	Set project goal and stakeholders	Goal from Section 1. Add official team members only after checking Moodle/tutor roster, IDs and group number. Identify the official group leader.
3	Create release / version	Release: R2 - English, Waste and Menu. Target window: 10 Nov to 21 Dec 2026 for implementation/evidence freeze; submission 22 Dec.
4	Create backlog hierarchy	Create the five Epics in Section 3, then Features if the chosen edition supports them. Add the user stories under each Epic. ScrumDesk documentation supports Epics, Features and User Stories; the depth varies by edition.
5	Set backlog item templates	Add fields to story template: ID, user story, outcome/value, priority/order, story points, acceptance criteria, dependencies, privacy/security notes, owner split, evidence link, and Done checklist.
6	Set workflow states	Use the states supported in the project: Backlog / Ready / In Progress / Review / Done / Blocked. If ScrumDesk edition uses different names, map these states and record the mapping.
7	Create team capacity	Enter actual member availability, absences and non-coding workload. Do not treat estimated story points as hours. Sabind codes; Harsha owns review/coordination/documentation tasks.
8	Create sprints	Create sprint dates and goal in PLAN. Pull ready items into the sprint, check capacity, then break items down into tasks in WORK. ScrumDesk’s current manual describes PLAN and WORK for planning and task tracking.
9	Set evidence conventions	Attach test output, screenshots, report excerpt, meeting minute, code/commit link, or measured metric directly to a card; link related evidence from the report.

ScrumDesk’s official manual describes backlog hierarchy, epics/features, story templates, ordering, sprint creation in PLAN, sprint goals/date fields, capacity indicators, task decomposition, and WORK-view task tracking. Menu names can vary with the edition/version. Follow the feature available in your actual account and capture a screenshot of the final board. [1, 2, 3]

3. Epic map and backlog order

Epic	Outcome / boundary	Target release
E1 - Bilingual Experience and Apple-Inspired UI	A user can choose English/Japanese before login and keep that language after login; every visible product string is translated; screens share a polished responsive visual system.	R2
E2 - Food Waste History	A user records discarded stock, sees their own history and waste summary, and keeps stock/history consistent.	R2
E3 - AI Menu Suggestions	A user gets recipe ideas in their selected language from their stock. Optional provider use is configured, transparent and resilient; local fallback works.	R2
E4 - Account and Profile Essentials	Registration, sign-in, editable display name/language, email identity and reset-by-email-code work securely.	R2
E5 - Quality, Release and Assessment Evidence	Testability, privacy/security, responsive regression, metrics, CI/repo evidence and report artifacts are ready for submission.	R2

Recommended top-to-bottom backlog order

- E1.1 Locale continuity and persistence; E1.2 complete English/Japanese coverage; E1.3 cohesive modern visual redesign.
- E4.1 registration/email/profile data; E4.2 profile and saved locale; E4.3 password-reset OTP and operational limits.
- E2.1 atomic waste recording; E2.2 history and totals; E2.3 ownership and empty/error states.
- E3.1 local inventory matching; E3.2 configured AI provider, language/expiry prompt, timeout and fallback; E3.3 privacy/user notice.
- E5.1 requirement-linked tests; E5.2 security and access isolation; E5.3 responsive/browser regression; E5.4 measured metrics, cost/COCOMO and submission evidence.

Prioritise by dependency and value, then order cards by drag-and-drop. ScrumDesk backlog listings can group or sort by Epic, theme, member, MoSCoW, business value, effort, state, readiness, sprint and release; use only fields available in the selected edition. [2]

4. Detailed Epic E1 stories

E1.1 Carry login language into the dashboard

> *As a user, I want the language I choose before signing in to remain selected after login so I do not have to reset the interface.*

Priority: Must **Estimate:** 5 points **Owner split:** Sabind codes; Harsha reviews scenarios/evidence
Depends on: Existing login and profile endpoints

Acceptance criteria

- Choosing English on login, then signing in, opens the dashboard in English.
- Choosing Japanese on login, then signing in, opens the dashboard in Japanese.
- The chosen language is saved in the account profile after successful authentication.
- If no login-page override was selected, the profile preference is used.
- A later login restores the saved preference.

Implementation and evidence tasks

Task	Type	Assigned
Track pending locale before authentication	Development	Sabind
Apply selected locale after auth and update profile	Development	Sabind

Task	Type	Assigned
Verify EN/JA sign-in scenarios; capture screen evidence	Testing	Harsha

E1.2 Translate all interface text and placeholders

As an English- or Japanese-speaking user, I want every control and message localized so no screen has accidental mixed-language UI.

Priority: Must **Estimate:** 5 points **Owner split:** Sabind codes; Harsha owns string inventory/review
Depends on: E1.1

Acceptance criteria

- Login, signup, recovery, dashboard, profile, filters, category labels, empty states, prompts, buttons, validation and status labels translate.
- HTML placeholders and aria labels change with the selected language.
- CSS-generated Japanese text is removed or supplied via translated markup.
- Browser date control follows the selected document language.
- User-entered food names/reasons remain unchanged.

Implementation and evidence tasks

Task	Type	Assigned
Inventory audit of visible strings and pseudo-content	Analysis	Harsha
Complete message catalogue and data-i18n bindings	Development	Sabind
Check all screens in both languages; record gaps	Testing	Harsha

E1.3 Apple-inspired responsive system UI

As a household member, I want a calm, consistent interface that is easy to scan on phone and desktop.

Priority: Should **Estimate:** 8 points **Owner split:** Sabind codes; Harsha reviews visual acceptance
Depends on: E1.2

Acceptance criteria

- Auth screens and dashboard use a consistent type scale, spacing, buttons, cards, focus states and color semantics.
- Inventory, AI ideas, waste history and profile remain usable at 360px width and desktop widths.
- Keyboard focus, labels, contrast and semantic headings are retained.
- No clipping, overlap, horizontal overflow or invisible actions in tested viewports.

Implementation and evidence tasks

Task	Type	Assigned
Define design tokens and reusable page/card states	Development	Sabind
Apply auth, profile, dashboard, inventory and history layout	Development	Sabind
Capture mobile/desktop screenshots; report accessibility issues	Review	Harsha

5. Detailed Epic E2 stories

E2.1 Record food waste transactionally

As a user, I want to record what I discard so the active inventory and waste history stay accurate.

Priority: Must **Estimate:** 8 points **Owner split:** Sabind codes; Harsha reviews scenarios
Depends on: Food and authenticated principal

Acceptance criteria

- A user can select an owned item, quantity and reason.
- Quantity must be positive and no greater than stock.
- Partial disposal decrements active quantity; full disposal removes the active food row.
- The event stores food name/category snapshot, discarded quantity, reason, time and owner.
- Event write and stock change commit together or roll back together.
- Invalid quantity or another user’s item returns a safe error and changes nothing.

Implementation and evidence tasks

Task	Type	Assigned
Review service transaction and ownership boundary	Development	Sabind
Validate quantity and reason; persist event and stock update	Development	Sabind
Test partial/full/invalid/foreign-owner/rollback cases	Testing	Harsha review; Sabind test code

E2.2 View waste history and summary

As a user, I want to see recent disposal records and total waste quantity so I can understand what is being lost.

Priority: Must **Estimate:** 5 points **Owner split:** Sabind codes; Harsha acceptance evidence
Depends on: E2.1

Acceptance criteria

- History is newest first and only includes the authenticated user’s events.
- Each row shows food name, quantity, reason and localized date.
- Dashboard total updates after recording waste and on page load.
- Empty state is localized and explains how history begins.
- No password, reset code or other private user fields appear in JSON.

Implementation and evidence tasks

Task	Type	Assigned
Provide and review user-scoped history API	Development	Sabind
Build history panel and summary metric	Development	Sabind
Verify empty/populated view and account isolation	Testing	Harsha

E2.3 Keep waste data isolated by account

As a user, I want my inventory and waste history private to my account.

Priority: Must **Estimate:** 3 points **Owner split:** Sabind codes; Harsha tracks security evidence
Depends on: E2.1, E2.2

Acceptance criteria

- All read/write paths derive identity from server authentication.

- Cross-account food/event requests are denied.
- Serialized waste responses do not expose the linked User entity or credentials.

Implementation and evidence tasks

Task	Type	Assigned
Review repository and controller scope	Security	Sabind
Add negative access tests and inspect JSON	Testing	Sabind implements; Harsha records results

6. Detailed Epic E3 stories

E3.1 Suggest meals from current stock

As a user, I want meal suggestions using what I already have so I can use food before it expires.

Priority: Must **Estimate:** 5 points **Owner split:** Sabind codes; Harsha acceptance review
Depends on: Inventory and E1 locale

Acceptance criteria

- Only positive-quantity items from the current user are considered.
- Up to five suggestions show name, description, matched inventory names, score and preparation time.
- Local fallback handles common English/Japanese names and empty or sparse stock.
- Nearest expiry may influence selection where dates are available.
- No inventory data is sent externally when AI settings are absent.

Implementation and evidence tasks

Task	Type	Assigned
Build local recipe/ingredient matcher and aliases	Development	Sabind
Render localized recommendation cards and empty state	Development	Sabind
Test matching, ranking and no-match cases	Testing	Harsha review

E3.2 Add an optional AI provider with resilient fallback

As a user, I want richer menu ideas from an AI provider when the project is configured for it.

Priority: Should **Estimate:** 8 points **Owner split:** Sabind codes; Harsha verifies privacy/acceptance
Depends on: E3.1, E1.1

Acceptance criteria

- Provider is enabled only by a non-empty secret and explicit endpoint/model config.
- Prompt requests output in the saved user language, uses inventory names/quantities/categories/expiry, and avoids unlisted ingredients except common basics.
- Expected response has bounded fields and maximum five entries.
- No-key, provider error, invalid response and timeout fall back to local ideas.
- API keys and inventory contents are not logged.

Implementation and evidence tasks

Task	Type	Assigned
Define environment-based provider config and timeout	Development	Sabind
Parse and bound response; catch provider and JSON errors	Development	Sabind

Task	Type	Assigned
Test absent key, valid stub, invalid response and timeout	Testing	Sabind implements; Harsha records

E3.3 Explain recommendation data use and limitations

As a user, I want to know whether my fridge inventory is sent to an external provider and what a suggestion means.

Priority: Must before AI enablement **Estimate:** 3 points **Owner split:** Sabind implements wording; Harsha coordinates review
Depends on: E3.2, E1.2

Acceptance criteria

- Local mode clearly states suggestions are based on available inventory.
- If remote AI is enabled, a localized disclosure states that inventory metadata is sent to the configured service.
- UI says suggestions are ideas and not allergy/nutrition or food-safety advice.
- Do not enable production remote mode before provider/privacy review.

Implementation and evidence tasks

Task	Type	Assigned
Write bilingual disclosure and suggestion limitation	Analysis / content	Harsha draft; Sabind binds to UI
Review provider data fields and configuration	Privacy	Harsha tracks; Sabind supplies technical detail
Capture disclosures in both languages	Review	Harsha

7. Detailed Epics E4 and E5 stories

E4.1 Register with email and a usable password

As a new user, I want to create an account with a recovery email so I can use my own fridge and recover access.

Priority: Must **Estimate:** 5 points **Owner split:** Sabind coding; Harsha requirements review
Depends on: Existing User entity and password encoder

Acceptance criteria

- Username is 3-40 characters and unique.
- Email has valid basic syntax, is normalized consistently and unique.
- Password minimum is eight characters and stored only as a one-way hash.
- Validation errors retain non-sensitive entered username/email.
- Legacy accounts without email are identified for migration/backfill before enabling their recovery.

Implementation and evidence tasks

Task	Type	Assigned
Extend/validate account model and service	Development	Sabind
Keep controller errors translated and form labels localized	Development	Sabind
Verify duplicates, invalid fields, password storage	Testing	Sabind tests; Harsha records

E4.2 Maintain profile name and language

As a signed-in user, I want my display name and language saved to my account.

Priority: Must **Estimate:** 3 points **Owner split:** Sabind coding; Harsha verifies
Depends on: E1.1, E4.1

Acceptance criteria

- Profile is scoped to current user.
- Display name is required and at most 60 characters.
- Only English/Japanese values accepted.
- Saved preference drives interface after next login unless user deliberately overrides at login.
- Email is read-only in this iteration.

Implementation and evidence tasks

Task	Type	Assigned
Validate and persist profile settings	Development	Sabind
Synchronize language control with profile drawer	Development	Sabind
Verify update and next-login behavior	Testing	Harsha

E4.3 Reset password through email OTP

As a registered user, I want a one-time code sent to my email so I can regain access securely.

Priority: Must **Estimate:** 8 points **Owner split:** Sabind codes; Harsha acceptance/security evidence
Depends on: E4.1, mail provider

Acceptance criteria

- Known and unknown addresses return the same confirmation text.
- Code is generated securely, stored hashed, expires after 10 minutes, and cannot be reused.

- Wrong/expired/used code is rejected; valid code updates hashed password.
- User is sent to normal login after reset; no automatic login.
- SMTP is environment-configured; dev-only log fallback is never described as production mail.
- Attempt and request throttling must be implemented before production release.

Implementation and evidence tasks

Task	Type	Assigned
Implement request/verify/reset flow and generic response	Development	Sabind
Configure SMTP through environment variables	Development / Ops	Sabind
Test delivery, generic response, expiry, replay and failure	Testing	Harsha records; Sabind implements tests

E5.1 Automate service and integration checks

As the project team, we want requirement-linked tests so regressions are detected before release.

Priority: Must **Estimate:** 8 points **Owner split:** Sabind implements all tests/coding; Harsha tracks/reviews
Depends on: E1-E4 feature completion

Acceptance criteria

- Unit tests cover waste quantity/ownership, menu matching/fallback, profile validation and OTP expiry/reuse.
- Integration tests cover registration, authenticated access, transaction behavior and serialized responses.
- Existing inventory CRUD/login flows have regression coverage.
- Maven build and test results are attached to stories and report traceability matrix.

Implementation and evidence tasks

Task	Type	Assigned
Add unit tests by feature service	Testing	Sabind
Add web/integration and ownership tests	Testing	Sabind
Review traceability links and evidence completeness	Review	Harsha

E5.2 Run responsive and localization regression

As an assessor/user, I want consistent interactions across key screens and languages.

Priority: Must **Estimate:** 5 points **Owner split:** Sabind fixes/code; Harsha executes and records
Depends on: E1.1-E1.3

Acceptance criteria

- Run login/signup/recovery/profile/dashboard/inventory/waste/menu at 360px and desktop.
- Switch languages on auth, then log in; profile and dashboard reflect selection.
- Buttons, date controls, alerts, table/card wrapping and focus states are usable.
- Attach dated screenshots and manual steps/results.

Implementation and evidence tasks

Task	Type	Assigned
Prepare EN/JA viewport script	Testing	Harsha
Fix defects	Development	Sabind
Record final screenshots and retest	Evidence	Harsha

E5.3 Collect metrics and freeze submission evidence

As the group, we want a defensible record of quality, cost and contribution for the report.

Priority: Must **Estimate:** 5 points **Owner split:** Harsha coordinates; Sabind supplies code/test evidence
Depends on: E5.1, E5.2

Acceptance criteria

- Record code size/count method, test pass rate, requirement coverage, endpoint timings and known defects.
- Recalculate COCOMO from measured enhancement size and state cost/rate assumptions.
- Report maps code, tests, backlog and contributions with stable evidence links.
- Attach repository URL, commits/branches/CI, ScrumDesk board/sprint evidence, dated meeting minutes, signed participation agreement and contribution matrix.
- Final report is 3,000 words +/-10%, contains COCOMO, PM docs, software metrics, cost estimation, testing, legal/ethical discussion and Harvard references (minimum 12 per supplied template).
- Each student completes the separate 10-tutorial document required by the brief.

Implementation and evidence tasks

Task	Type	Assigned
Collect baseline and post-change data	Metrics	Sabind technical measures; Harsha records
Complete report, bibliography and contribution appendix	Documentation	Harsha coordinates; Sabind technical input
Check submission links, cover sheet and deadline	Release	Official group leader (confirm)

8. Sprint plan and planning workshop

Use these as provisional dates to enter; adjust to actual team capacity and tutorial feedback. ScrumDesk's sprint form supports a name, goal, start date and end date; sprint planning is performed in PLAN, and selected stories are decomposed into tasks for WORK. Check the sprint capacity indicators rather than pulling every card just because it is in the backlog. [1, 4]

Sprint/window	Sprint Goal	Candidate stories	Review exit
S1 | 10-23 Nov	Make language and account UX coherent from auth through dashboard.	E1.1, E1.2, E1.3, E4.2	Both locales verified; modern visual screens reviewed; profile persistence evidence attached.
S2 | 24 Nov-7 Dec	Record and review disposal safely.	E2.1, E2.2, E2.3, E4.1	Partial/full waste and ownership evidence; account registration scenarios pass.
S3 | 8-15 Dec	Deliver resilient menu ideas and complete reset security.	E3.1, E3.2, E3.3, E4.3	Local and configured AI/fallback cases; OTP expires/consumes; privacy disclosure reviewed.
Hardening | 16-20 Dec	Close release quality and assessment evidence.	E5.1, E5.2, E5.3; all defect stories	Regression/metrics/report/board evidence reviewed; unresolved defects listed.
Submission prep | 21 Dec	Freeze approved files and confirm submission handoff.	Final report and tutorial/participation artifacts	Official group leader verifies upload/package and acknowledges to group.

During sprint planning, create tasks normally no longer than one working day. Give each task an action title, type, description, assignee, estimate/remaining/spent time as used by the group, dependency/blocker and evidence attachment. ScrumDesk WORK can show a backlog item with its subtasks and track task types, timing, waiting/impediment flags and assignees. [1]

Planning meeting script

- Confirm product goal, current increment, previous unfinished cards and impediments.
- Choose top ready stories in backlog order; read acceptance criteria aloud and clarify ambiguity.
- Estimate relatively (Fibonacci scale); split stories too large to complete and review in one sprint.
- Break each story into coding, test, localization, review and evidence tasks; assign Sabind to coding and Harsha to non-coding work.
- Check real availability, tutorials/lectures, report workload and absences; adjust scope.
- Agree sprint goal, dependencies, risks, review evidence and board state; record decisions and minutes.

9. Daily work and board workflow

State	Entry rule	Exit rule / evidence
Backlog	Idea has ID and Epic.	Ranked, value/scope understood.
Ready	Story has acceptance criteria, priority, estimate, owner, dependencies and feasible tasks.	Team selects it during planning.
In Progress	Work has started; task moved and active owner visible.	Implementation complete and self-checked; code link exists.
Review	Code is ready for technical/acceptance/security review.	Reviewer evidence attached; defects returned to In Progress.
Done	Every acceptance criterion passes, security/ownership considered, tests pass, translation and responsive checks pass, evidence attached.	No further work required for release.
Blocked	External dependency or obstacle prevents progress; reason and owner/due date recorded.	Blocker resolved and item returned to its prior active state.

Daily 10-15 minute sync: what moved since last check, next task, blocker, and current remaining work. This is team coordination, not a status presentation. Harsha keeps ScrumDesk current and records impediments; Sabind reports coding/test status and moves task cards. Update remaining time honestly. A waiting/impediment flag needs a comment with cause, owner and next action.

Meeting minutes (mandatory assessment evidence)

For every team meeting, log date/time/venue or call link, members present, apologies, decisions, work allocated to each person by next meeting, next meeting time/date/venue, and approval/signatures. Keep minutes factual and contemporaneous. A planning guide or an unsigned retrospective note does not replace the required signed group participation agreement. [5]

10. Sprint review, retrospective and release closure

Sprint Review in ScrumDesk

- Open the sprint and review each selected story against acceptance criteria; demonstrate the feature in the running app.
- Attach test logs, issue references, screen captures and any change in scope. Show the Done/Incomplete split.
- Capture ScrumDesk sprint report with story statuses and effort. ScrumDesk manual describes sprint report fields for items and status; use the report available in the team edition. [5]
- Move incomplete stories back to backlog or a later sprint and re-estimate/reorder with team discussion. Do not mark partially implemented cards Done.

Retrospective

- Ask what helped, what caused delays/defects, and what the team will try next sprint.
- Choose one or two measurable improvement actions with owner and date.
- Record actions in ScrumDesk or linked meeting minutes; follow up at the next retrospective.

Release and assessment close-out checklist

Check	ScrumDesk / submission evidence
Features complete	All Must stories Done; Should stories accepted or explicitly deferred; no hidden blockers.
Quality	Java 21 build/test run; results and defects attached; manual EN/JA and responsive flows checked.
Data and security	Ownership tests, OTP expiry/reuse, privacy notes, secrets handling, external AI setting and fallback documented.
Metrics and cost	Baseline/post-change measure method/results; measured KLOC; COCOMO recalculation; assumptions and cost rate clear.
Version control / CI	Repository accessible to tutor/module leader; dated branches/commits show collaboration; CI build/test result attached.
Project report	3,000 words +/-10%; use supplied template; analysis, architecture/UML/use cases, testing, COCOMO, PM, metrics, cost, legal/ethical and Harvard references.
Individual tutorials	Each student submits the required ten tutorial tasks document separately as described in brief.
Contributions	Who-did-what table with actual task/hour evidence; each member participation agreement completed and signed.
Submission	Cover sheet, source code/asset link or complete source archive, report PDF/DOCX and confirmation from official group leader by 22 Dec 2026 23:59.

11. Roles and responsibility split

Work	Sabind Daryal	Harsha
Backend/frontend/data model/integration	Sole coding owner	No coding assigned
Automated tests and defect fixes	Implements test code and fixes	Reviews requirements/test evidence; records result
Requirements/story acceptance	Technical input	Maintains criteria, backlog and review evidence
ScrumDesk board	Supplies code status; links commits	Board setup, ordering, sprint records, screenshots
Minutes and contribution log	Reports own activity truthfully	Maintains dated minutes, hours and evidence for all members
Report and submission	Technical architecture/code/test input	Coordinates report, references, appendices and checklist

Do not claim equal contribution just because a planned allocation says so. Enter actual hours and work evidence. The two named-member plan conflicts with the supplied brief's 3-4 person team condition; confirm group membership, group leader and contribution expectations with the tutor.

12. Metrics and definition of ready/done

Definition of Ready: user value is stated; story is correctly nested under an Epic; acceptance criteria are testable; scope is small; language/security/privacy implications and dependencies are explicit; estimate and priority/order exist; test/evidence approach is known; owner split is possible.

Definition of Done: feature code is reviewed; relevant automated tests pass; all acceptance criteria pass; ownership and error handling are checked; English/Japanese strings, placeholders and accessible names are complete; responsive screenshots are clean; secrets are not exposed; report/traceability and board links are updated. For optional remote AI, provider failure fallback and data disclosure also pass.

Measure	How to collect in ScrumDesk / repo	Release target
Requirement coverage	Map story IDs to requirement IDs and test IDs; count covered Must requirements.	100% Must stories have test evidence or approved defer reason.
Test pass rate	Attach Maven test report; pass / executed.	100% for release-gate tests; report skipped tests honestly.
Waste correctness	Scenario checks for partial/full quantity, rollback and owner isolation.	All mandatory scenarios pass.
Localization gaps	Checklist of screens/placeholders/validation/dynamic states in EN and JA.	No unexplained untranslated UI strings.
Performance	Time local menu/history APIs with fixed data set; report p50/p95 and machine.	Target in project pack; record measured value and limits.
Cost/size	Measure net new source LOC and actual logged effort; calculate COCOMO assumptions.	No fabricated actual cost; separate forecast from actual.

Appendix A. Ready-to-copy story template

ID and title: ____
Epic / feature: ____
User story: As a [user], I want [capability] so that [value].
Priority/order: ____ **Estimate (story points):** ____ **Release/sprint:** ____
Acceptance criteria: Given/When/Then statements with observable outcomes.
Dependencies: ____ **Risks/privacy/security:** ____
Owner split: Coding: Sabind; review/evidence/board: Harsha; update if official roster differs.
Tasks: one action, type, assignee, estimate, acceptance/evidence link each; coding/test tasks assigned only per agreed split.
Evidence links: commit/PR ____; test output ____; screenshot ____; report section ____.
Done checklist: code/review, tests, ownership, bilingual states, responsive review, evidence, known issues.

Appendix B. Sprint planning and close-out record template

Field	Entry
Date/time/venue	
Sprint name / dates / goal	
Present / apologies	
Capacity/availability assumptions	
Stories pulled / story points	
Tasks, owner, estimate and evidence	
Dependencies / impediments and next action	
Decisions / scope changes	
Next meeting time, date and venue	
Agreement / signatures	

Appendix C. Source notes and references

This implementation playbook is a planning artifact, not a record of a live ScrumDesk board. Access the current workspace and verify account-specific UI/edition names before entering records. Access dates below are 10 November 2026.

[1] ScrumDesk (2026) *Plan sprint*. ScrumDesk Start Manual. Available at: https://www.scrumdesk.com/start/manual-for-scrumdesk-start/start-plan-sprint/ (Accessed: 10 November 2026).

[2] ScrumDesk (2026) *Backlog management*. ScrumDesk Start Manual. Available at: https://www.scrumdesk.com/start/manual-for-scrumdesk-start/start-backlog-management/ (Accessed: 10 November 2026).

[3] ScrumDesk (2026) *Product videos: backlog hierarchy, stories, releases and sprint planning*. Available at: https://www.scrumdesk.com/product-videos/ (Accessed: 10 November 2026).

[4] ScrumDesk (2026) *Proper sprint planning in Scrum*. Available at: https://www.scrumdesk.com/start/manual-for-scrumdesk-start/sprint-planning/ (Accessed: 10 November 2026).

[5] University of East London (2026) *CN7021 Advanced Software Engineering Coursework Brief 2026-27*. Supplied PDF, pp. 3-8.

[6] University of East London (2025) *CN7021 ASWE Project Report Template*. Supplied DOCX, 2024-25 version.

[7] Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide*. Available at: https://scrumguides.org/scrum-guide.html (Accessed: 10 November 2026).